Boiler Room Labb 1
Spring Boot, Lagerarkitektur och Databas

YH Backend

Syfte

Denna labb handlar inte om funktionalitet. Den handlar om struktur.

Ni ska bygga en minimal Spring Boot-applikation som:

- Sparar ett objekt i databasen
- Hämtar objekt från databasen
- Returnerar dem som JSON

Det viktiga är att ni bygger hela lagerkedjan korrekt och förstår vad som händer i varje steg.

Arkitekturöversikt

Flödet i applikationen:

$$\text{HTTP} \rightarrow \text{Controller} \rightarrow \text{Service} \rightarrow \text{Repository} \rightarrow \text{Databas}$$

och tillbaka samma väg.

Varför lager?

Vi bygger system som ska:

- Växa
- Testas
- Underhållas
- Skalera

Separation of concerns gör system:

- Testbara
- Begripliga
- Förändringsbara

Steg 1 – Skapa projekt (exakt så här)

Gå till:
`https://start.spring.io`

Inställningar

- Project: Maven
- Language: Java
- Spring Boot: Senaste stabila (ej SNAPSHOT)
- Group: com.example
- Artifact: boilerroom-labb1
- Name: boilerroom-labb1
- Packaging: Jar
- Java: 17 eller 21

Dependencies

- Spring Web
- Spring Data JPA
- H2 Database

Generera projektet och öppna som Maven-projekt i IDE.
Starta applikationen.
Gå till:
`http://localhost:8080`
404 är korrekt.

Steg 2 – Skapa paket och filer (mycket noggrant)

Under:
`src/main/java/com.example.boilerroom`
Skapa packages:

- controller
- service
- repository
- entity

I IDE:

- Högerklicka på com.example.boilerroom
- New → Package

Strukturen ska bli:

```
com.example.boilerroom
     controller
     service
     repository
     entity
```

Skillnad mellan klass och interface

Klass:

- Innehåller implementation
- Skapar objekt

Interface:

- Definierar ett kontrakt
- Ingen implementation
- Spring skapar implementation vid runtime

Repository är ett interface.

Steg 3 – Entity (klass)

Skapa i package `entity`:
New → Java Class → Message

```
package com.example.boilerroom.entity;

import jakarta.persistence.*;

@Entity
public class Message {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String text;
    private int number;

    public Message() {}

    public Message(String text, int number) {
        this.text = text;
        this.number = number;
    }

    public Long getId() { return id; }
    public String getText() { return text; }
    public int getNumber() { return number; }

    public void setText(String text) { this.text = text; }
    public void setNumber(int number) { this.number = number; }
}
```

@Entity gör klassen till en databastabell.

Steg 4 – Repository (interface)

Skapa i package `repository`:

New → Java Interface → MessageRepository

```
package com.example.boilerroom.repository;

import com.example.boilerroom.entity.Message;
import org.springframework.data.jpa.repository.JpaRepository;

public interface MessageRepository
        extends JpaRepository<Message, Long> {
}
```

Spring genererar implementation automatiskt.

Steg 5 – Service (klass)

Skapa i package `service`:

New → Java Class → MessageService

```
package com.example.boilerroom.service;

import com.example.boilerroom.entity.Message;
import com.example.boilerroom.repository.MessageRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class MessageService {

    private final MessageRepository repository;

    public MessageService(MessageRepository repository) {
        this.repository = repository;
    }

    public Message create(String text, int number) {
        Message message = new Message(text, number);
        return repository.save(message);
    }

    public List<Message> getAll() {
        return repository.findAll();
    }
}
```

@Service gör klassen till en Spring Bean.

Steg 6 – Controller (klass)

Skapa i package `controller`:

New → Java Class → MessageController

```
package com.example.boilerroom.controller;

import com.example.boilerroom.entity.Message;
import com.example.boilerroom.service.MessageService;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/messages")
public class MessageController {

    private final MessageService service;

    public MessageController(MessageService service) {
        this.service = service;
    }

    @PostMapping
    public Message create() {
        return service.create("Hej␣YH", 42);
    }

    @GetMapping
    public List<Message> getAll() {
        return service.getAll();
    }
}
```

HTTP-request → Controller → Service → Repository.

Steg 7 – application.properties (Konfiguration)

I projektet finns mappen:
`src/main/resources`
Där finns filen:
`application.properties`
Detta är applikationens konfigurationsfil.
Här styr vi:

- Databasanslutning
- Hibernate-inställningar
- Serverinställningar

Lägg till följande:

```
spring.h2.console.enabled=true
spring.datasource.url=jdbc:h2:mem:testdb
spring.jpa.hibernate.ddl-auto=update
```

Vad betyder detta?

spring.h2.console.enabled=true
Aktiverar H2:s webbkonsol så vi kan titta på databasen i webbläsaren.

spring.datasource.url=jdbc:h2:mem:testdb

- jdbc:h2 → vi använder H2-databasen
- mem → databasen ligger i minnet
- testdb → namnet på databasen

Det betyder:
Databasen finns bara så länge applikationen kör.
När ni stoppar servern försvinner all data.

spring.jpa.hibernate.ddl-auto=update
Detta talar om för Hibernate:

- Skapa tabeller automatiskt
- Uppdatera tabellstruktur om Entity ändras

Utan denna rad skulle tabellen inte skapas automatiskt.

Viktigt

application.properties är inte kod.
Det är konfiguration.
Den påverkar hur Spring Boot startar och hur ramverket beter sig.

Hur kontrollerar vi att allt fungerar? (med HTTPie)

Vi använder verktyget **HTTPie** för att testa vårt API.

Om HTTPie inte är installerat:

```
pip install httpie
```

eller via brew:

```
brew install httpie
```

Kontrollera installation:

```
http --version
```

1. Starta applikationen

Starta Spring Boot-applikationen.

I konsolen ska ni se något liknande:

```
Tomcat started on port 8080
Started BoilerroomLabb1Application
```

Om applikationen inte startar korrekt – lös det först.

2. Skapa ett objekt (POST)

Öppna terminalen och kör:

```
http POST localhost:8080/messages
```

Ni ska få ett JSON-svar som liknar:

```
{
  "id": 1,
  "text": "Hej YH",
  "number": 42
}
```

Detta betyder att:

- Controller tog emot requesten
- Service skapade objektet
- Repository sparade det
- Databasen lagrade det

3. Hämta alla objekt (GET)

Kör:

```
http GET localhost:8080/messages
```

Ni ska få en lista:

```
[
  {
    "id": 1,
    "text": "Hej YH",
    "number": 42
  }
]
```

4. Kontrollera databasen via H2

Öppna webbläsaren:
`http://localhost:8080/h2-console`
Ange:

- JDBC URL: jdbc:h2:mem:testdb
- User: sa
- Password: (lämna tomt)

Klicka Connect.
Kör SQL:

```
SELECT * FROM MESSAGE;
```

Om ni ser data där fungerar hela flödet:

$$\text{HTTP} \rightarrow \text{Controller} \rightarrow \text{Service} \rightarrow \text{Repository} \rightarrow \text{Databas}$$

Felsökning

Om POST inte fungerar:

- Kontrollera att applikationen kör
- Kontrollera att port 8080 används
- Kontrollera att URL är korrekt

Om GET returnerar tom lista:

- Har ni gjort POST först?
- Startade ni om servern (H2 är in-memory)?

Kontrollfrågor – Förstå vad som händer

Diskutera i grupp:

1. Vad händer tekniskt när applikationen startar?
2. När skapas databastabellen?
3. Vad händer när ni skickar en POST-request?
4. Vem skapar Message-objektet?
5. Vem anropar repository.save()?
6. Hur vet Spring vilken Service som ska kopplas till Controller?
7. Vad händer om ni tar bort @Service?
8. Vad händer om ni tar bort @Entity?
9. Varför försvinner datan när ni stoppar servern?

Djupare förståelse

Försök beskriva hela flödet:

$$\text{Klient} \rightarrow \text{HTTP} \rightarrow \text{Controller} \rightarrow \text{Service} \rightarrow \text{Repository} \rightarrow \text{Hibernate} \rightarrow \text{Databas}$$

och tillbaka.
Om ni kan förklara detta utan att titta i koden har ni förstått grunden.

Utmaning – Bygg vidare själv

Nu fungerar grundversionen.

Ni ska nu vidareutveckla applikationen utan fullständig steg-för-steg-instruktion.

Detta är en förståelseövning.

Del 1 – Gör POST dynamisk

Just nu är POST hårdkodad.

Ert mål:

- Ta bort hårdkodade värden
- Låt klienten skicka in text och number via HTTP

Ni ska kunna göra:

```
http POST localhost:8080/messages text="Hej igen" number=99
```

Tips:

- Ni behöver ändra i Controller
- Titta på annotationen `@RequestBody`
- Fundera på om ni vill använda Entity direkt eller skapa en ny klass

Ni får använda:

- Dokumentation
- Google
- AI

Men ni ska förstå vad koden gör.

Del 2 – Hämta bara number

Skapa en ny endpoint som returnerar endast number för ett specifikt id.

Exempel:

```
http GET localhost:8080/messages/1/number
```

Tips:

- Ni behöver en ny `@GetMapping`
- Ni behöver använda `@PathVariable`
- Ni behöver anropa Service-lagret

Fundera på:

- Ska ni returnera ett rått int?
- Eller ett JSON-objekt?

Del 3 – Hämta bara text

Skapa motsvarande endpoint:

```
http GET localhost:8080/messages/1/text
```

Reflektion

När ni är klara, diskutera:

1. Vilka filer behövde ni ändra?
2. Behövde Service ändras?
3. Behövde Repository ändras?
4. Varför eller varför inte?
5. Är det bra design att exponera varje fält som egen endpoint?

Djupare designfråga

Anta att Message senare får 20 fält.
Ska ni skapa 20 endpoints?
Vad säger det om API-design?

Appendix A – Hur Spring använder Annotations

1. Vad händer när applikationen startar?

När ni startar applikationen händer följande:

1. Spring Boot startar en ApplicationContext
2. Spring skannar projektet efter klasser med särskilda annotations
3. Spring skapar objekt (Beans) av dessa klasser
4. Spring kopplar ihop dem via Dependency Injection
5. En embedded webbserver (Tomcat) startas

Detta kallas **Component Scanning**.
Spring letar automatiskt efter klasser i samma paket och underpaket som huvudklassen (den med `@SpringBootApplication`).

2. Vad är en Bean?

En **Bean** är ett objekt som hanteras av Spring.
Istället för att ni skriver:

```
MessageService service = new MessageService();
```

så skapar Spring objektet åt er och placerar det i sin container (ApplicationContext).
När en klass behöver en annan klass, injicerar Spring den automatiskt via konstruktorn.

3. Vad är Dependency Injection?

Dependency Injection betyder:
En klass skapar inte sina egna beroenden. De tilldelas utifrån.
Exempel:

```
public MessageService(MessageRepository repository) {
    this.repository = repository;
}
```

Spring ser:

- MessageService behöver MessageRepository
- Det finns en Bean av typen MessageRepository
- Spring kopplar ihop dem automatiskt

Vanliga Annotations i denna labb

@SpringBootApplication

Sitter på huvudklassen.

Den kombinerar tre saker:

- @Configuration
- @EnableAutoConfiguration
- @ComponentScan

Detta gör att:

- Spring laddar konfiguration
- Spring aktiverar auto-konfiguration
- Spring börjar scanna efter komponenter

@Entity

Markerar en klass som en JPA-entitet.

Hibernate använder den för att:

- Skapa en databastabell
- Mappa klassens fält till kolumner

@Id

Markerar primärnyckeln i tabellen.

@GeneratedValue

Talar om hur id ska genereras (t.ex. auto-increment).

@Repository

Markerar ett repository-lager.

I denna labb används inte annotationen direkt eftersom `JpaRepository` redan är en specialiserad Spring-komponent.

Spring Data genererar implementation automatiskt.

@Service

Markerar klassen som service-lager.

Spring skapar en Bean av klassen och gör den tillgänglig för injection.

@RestController

Kombinerar:

- @Controller
- @ResponseBody

Det betyder att:

- Klassen hanterar HTTP-requests
- Returvärden konverteras automatiskt till JSON

@RequestMapping

Definierar en bas-URL för alla metoder i klassen.
Exempel:

```
@RequestMapping("/messages")
```

@GetMapping

Binder en metod till HTTP GET.

@PostMapping

Binder en metod till HTTP POST.

@PathVariable

Används för att läsa värden från URL:en.
Exempel:

```
/messages/{id}
```

@RequestBody

Talar om för Spring att:

- Läsa HTTP-requestens body
- Konvertera JSON till ett Java-objekt

Spring använder biblioteket Jackson för konverteringen.

Vad betyder Auto-Configuration?

Spring Boot försöker gissa vad ni behöver baserat på:

- Vilka dependencies som finns i pom.xml
- Vilka klasser som finns i projektet

Exempel:
Om Spring hittar:

- H2 dependency
- JPA dependency

Då konfigureras:

- Datasource
- Hibernate
- EntityManager

utan att ni skriver konfigurationskod.

Sammanfattning

Annotations i Spring är metadata.
De talar om för ramverket:

- Vad som är en komponent
- Vad som är en entitet
- Vad som är en endpoint
- Hur objekt ska kopplas ihop

Spring använder reflection och component scanning för att läsa dessa och bygga upp applikationen dynamiskt vid start.

Appendix B – Vad gör Spring?

- Skannar klasser efter annotations
- Skapar Beans i ApplicationContext
- Injicerar beroenden
- Startar embedded server

Appendix C – H2 Databasen

Vad är H2?

H2 är en lättvikts relationsdatabas skriven i Java.

Den kan köras på två sätt:

- In-memory (lagras i RAM)
- Filbaserad (lagras på disk)

I denna labb använder vi:

```
jdbc:h2:mem:testdb
```

Det betyder att databasen:

- Skapas när applikationen startar
- Ligger i minnet
- Försvinner när applikationen stoppas

Varför använder vi H2 i kursen?

Vi använder H2 för att:

- Slippa installera extern databas
- Säkerställa att alla studenter har samma miljö
- Kunna fokusera på arkitektur och JPA

Det gör att vi kan träna på:

- Entity-mappning
- Repository-lagret
- Hibernate
- SQL-generering

utan att hantera databasinstallation och drift.

Är H2 en produktionsdatabas?

Nej.

I verkliga system används databaser som:

- PostgreSQL
- MySQL
- Oracle
- SQL Server

H2 används ofta i:

- Tester
- Prototyper
- Utbildning

In-memory vs produktionsdatabas

In-memory-databas:

- Data försvinner vid omstart
- Kräver ingen installation
- Snabb att starta

Produktionsdatabas:

- Lagrar data permanent
- Körs ofta på separat server
- Kräver konfiguration och säkerhetshantering

Appendix D – Ordlista

JPA – Java Persistence API, standard för objekt-databas-mappning.
Hibernate – Implementation av JPA.
CRUD – Create, Read, Update, Delete.
REST – Arkitekturstil för webb-API.
HTTP – Kommunikationsprotokoll för webb.
JSON – Textformat för datautbyte.
Dependency Injection – Ramverket tilldelar beroenden.
Bean – Objekt hanterat av Spring.
Maven – Byggverktyg för Java.
Jar – Körbar Java-arkivfil.